

Service Worker API

Note: This feature is available in [Web Workers](#).

Service workers essentially act as proxy servers that sit between web applications, the browser, and the network (when available). They are intended, among other things, to enable the creation of effective offline experiences, intercept network requests, and take appropriate action based on whether the network is available, and update assets residing on the server. They will also allow access to push notifications and background sync APIs.

Note: Service workers are a type of web worker. See [Web workers](#) for general information about worker types and use cases.

Service worker concepts and usage

A service worker is an event-driven [worker](#) registered against an origin and a path. It takes the form of a JavaScript file that can control the web page/site that it is associated with, intercepting and modifying navigation and resource requests, and caching resources in a very granular fashion to give you complete control over how your app behaves in certain situations (the most obvious one being when the network is not available).

Service workers run in a worker context: they therefore have no DOM access and run on a different thread to the main JavaScript that powers your app. They are non-blocking and designed to be fully asynchronous. As a consequence, APIs such as synchronous [XHR](#) and [Web Storage](#) can't be used inside a service worker.

Service workers can't import JavaScript modules dynamically, and `import()` will throw an error if it is called in a service worker global scope. Static imports using the `import` statement are allowed.

Service workers are only available in [secure contexts](#): this means that their document is served over HTTPS, although browsers also treat `http://localhost` as a secure context, to facilitate local development. HTTP connections are susceptible to malicious code injection by [man in the middle](#) attacks, and such attacks could be worse if allowed access to these powerful APIs.

Note: On Firefox, for testing you can run service workers over HTTP (insecurely); simply check the **Enable Service Workers over HTTP (when toolbox is open)** option in the Firefox DevTools options/gear menu.

Note: Unlike previous attempts in this area such as [AppCache](#), service workers don't make assumptions about what you are trying to do, but then break when those assumptions are not exactly right. Instead, service workers give you much more granular control.

Note: Service workers make heavy use of [promises](#), as generally they will wait for responses to come through, after which they will respond with a success or failure action. The promises architecture is ideal for this.

Registration

A service worker is first registered using the `ServiceWorkerContainer.register()` method. If successful, your service worker will be downloaded to the client and attempt installation/activation (see below) for URLs accessed by the user inside the whole origin, or a subset specified by you.

Download, install and activate

At this point, your service worker will observe the following lifecycle:

1. Download
2. Install
3. Activate

The service worker is immediately downloaded when a user first accesses a service worker-controlled site/page.

After that, it is updated when:

- A navigation to an in-scope page occurs.
- An event is fired on the service worker and it hasn't been downloaded in the last 24 hours.

Installation is attempted when the downloaded file is found to be new — either different to an existing service worker (byte-wise compared), or the first service worker encountered for this page/site.

If this is the first time a service worker has been made available, installation is attempted, then after a successful installation, it is activated.

If there is an existing service worker available, the new version is installed in the background, but not yet activated — at this point it is called the *worker in waiting*. It is only activated when there are no longer any pages loaded that are still using the old service worker. As soon as there are no more pages to be loaded, the new service worker activates (becoming the *active worker*). Activation can happen sooner using `ServiceWorkerGlobalScope.skipWaiting()` and existing pages can be claimed by the active worker using `Clients.claim()`.

You can listen for the `install` event; a standard action is to prepare your service worker for usage when this fires, for example by creating a cache using the built-in storage API, and placing assets inside it that you'll want for running your app offline.

There is also an `activate` event. The point where this event fires is generally a good time to clean up old caches and other things associated with the previous version of your service worker.

Your service worker can respond to requests using the `FetchEvent` event. You can modify the response to these requests in any way you want, using the `FetchEvent.respondWith()` method.

Note: Because `install` / `activate` events could take a while to complete, the service worker spec provides a `waitUntil()` method. Once it is called on `install` or `activate` events with a promise, functional events such as `fetch` and `push` will wait until the promise is successfully resolved.

For a complete tutorial to show how to build up your first basic example, read [Using Service Workers](#).

Using static routing to control how resources are fetched

Service workers can incur an unnecessary performance cost — when a page is loaded for the first time in a while, the browser has to wait for the service worker to start up and run to know what content to load and whether it should come from a cache or the network.

If you already know ahead of time where certain content should be fetched from, you can bypass the service worker altogether and fetch resources immediately. The `InstallEvent.addRoutes()` method can be used to implement this use case and more.

Other use case ideas

Service workers are also intended to be used for such things as:

- Background data synchronization.
- Responding to resource requests from other origins.
- Receiving centralized updates to expensive-to-calculate data such as geolocation or gyroscope, so multiple pages can make use of one set of data.
- Client-side compiling and dependency management of CoffeeScript, less, CJS/AMD modules, etc. for development purposes.
- Hooks for background services.
- Custom templating based on certain URL patterns.
- Performance enhancements, for example, pre-fetching resources that the user is likely to need soon, such as the next few pictures in a photo album.
- API mocking.

In the future, service workers will be able to do several other useful things for the web platform that will bring it closer to native app viability. Interestingly, other specifications can and will start to make use of the service worker context, for example:

- [Background synchronization](#) : Start up a service worker even when no users are at the site, so caches can be updated, etc.
- [Reacting to push messages](#): Start up a service worker to send users a message to tell them new content is available.
- Reacting to a particular time & date.
- Entering a geo-fence.

Interfaces

Cache

Represents the storage for `Request` / `Response` object pairs that are cached as part of the `ServiceWorker` life cycle.

CacheStorage

Represents the storage for `Cache` objects. It provides a master directory of all the named caches that a `ServiceWorker` can access, and maintains a mapping of string names to corresponding `Cache` objects.

Client

Represents the scope of a service worker client. A service worker client is either a document in a browser context or a `SharedWorker`, which is controlled by an active worker.

Clients

Represents a container for a list of `Client` objects; the main way to access the active service worker clients at the current origin.

ExtendableEvent

Extends the lifetime of the `install` and `activate` events dispatched on the `ServiceWorkerGlobalScope`, as part of the service worker lifecycle. This ensures that any functional events (like `FetchEvent`) are not dispatched to the `ServiceWorker`, until it upgrades database schemas, and deletes outdated cache entries, etc.

`ExtendableMessageEvent`

The event object of a `message` event fired on a service worker (when a channel message is received on the `ServiceWorkerGlobalScope` from another context) — extends the lifetime of such events.

`FetchEvent`

The parameter passed into the `onfetch` handler, `FetchEvent` represents a fetch action that is dispatched on the `ServiceWorkerGlobalScope` of a `ServiceWorker`. It contains information about the request and resulting response, and provides the `FetchEvent.respondWith()` method, which allows us to provide an arbitrary response back to the controlled page.

`InstallEvent`

The parameter passed into an `install` event handler function, the `InstallEvent` interface represents an install action that is dispatched on the `ServiceWorkerGlobalScope` of a `ServiceWorker`. As a child of `ExtendableEvent`, it ensures that functional events such as `FetchEvent` are not dispatched during installation.

`NavigationPreloadManager`

Provides methods for managing the preloading of resources with a service worker.

`ServiceWorker`

Represents a service worker. Multiple browsing contexts (e.g., pages, workers, etc.) can be associated with the same `ServiceWorker` object.

`ServiceWorkerContainer`

Provides an object representing the service worker as an overall unit in the network ecosystem, including facilities to register, unregister, and update service workers, and access the state of service workers and their registrations.

`ServiceWorkerGlobalScope`

Represents the global execution context of a service worker.

`ServiceWorkerRegistration`

Represents a service worker registration.

`WindowClient`

Represents the scope of a service worker client that is a document in a browser context, controlled by an active worker. This is a special type of `Client` object, with some additional methods and properties available.

Extensions to other interfaces

`Window.caches` and `WorkerGlobalScope.caches`

Returns the `CacheStorage` object associated with the current context.

`Navigator.serviceWorker` and `WorkerNavigator.serviceWorker`

Returns a `ServiceWorkerContainer` object, which provides access to registration, removal, upgrade, and communication with the `ServiceWorker` objects for the [associated document](#) [↗](#).

Specifications

Specification

[Service Workers Nightly](#) 

See also

- [Using Service Workers](#)
- [Service Worker Lifecycle](#) 
- [Service workers basic code example](#) 
- Web APIs that are related to the Service Worker API:
 - [Background Fetch API](#)
 - [Background Synchronization API](#)
 - [Content Index API](#)
 - [Cookie Store API](#)
 - [Notifications API](#)
 - [Web-based Payment Handler API](#)
 - [Push API](#)
 - [Web Periodic Background Synchronization API](#)



Your blueprint for a better internet.